

第50回技能五輪全国大会 「電子機器組立て」職種

競技Ⅱ

別添資料

- ・「電子はかり」取扱説明書
- ・「電子はかり」回路図
- ・「電子はかり」部品表
- ・「電子はかり」基板図
- ・「電子はかり」組立図
- ・「電子はかり」C言語プログラムソースリスト

第50回技能五輪全国大会
「電子機器組立て」職種

競技Ⅱ

「電子はかり」取扱説明書

1. 機能

1.1 各部の名称と機能

図 1.1 に「電子はかり」の写真と各部の名称を示します。



図 1.1 「電子はかり」外観

各部の機能は、以下の通りです。

- (1) 電源スイッチ： 電源のオン、オフを切り替えます。
- (2) 計量皿： 計量したいものを載せる部分です。
- (3) スイッチ A： 「電子はかり」を操作するためのスイッチです。
- (4) スイッチ B： 「電子はかり」を操作するためのスイッチです。
- (5) スイッチ C： 「電子はかり」を操作するためのスイッチです。
- (6) LCD 表示器： 測定結果や操作に対する指示等を表示します。1 行目には主に測定結果等、2 行目には主にメニュー等が表示されます。
- (7) リセットスイッチ： PIC をリセットするためのスイッチです。
- (8) モジュラジャック： PIC にプログラムを書き込むために、マイクロチップ社の ICD2, ICD3, PICkit2, または PICkit3 を接続するためのジャックです。
- (9) 電池ボックス： 006P 型 9V アルカリ乾電池を取り付ける部分です。
- (10) 半固定抵抗器 VR51： LCD 表示器のコントラストを調整します。
- (11) 半固定抵抗器 VR1, VR2： 回路定数を調整します。

2. 注意事項

2.1 一般使用における注意事項

- (1) 使用する前に電源スイッチをオフにした状態で、電池を電池ボックスに取り付けてください。
- (2) 電池は指定されたものを使用してください。
- (3) 電池の極性を誤って接続すると回路破損の原因となります。
- (4) 平らな安定している場所に設置してください。斜めに設置したり、がたつきがあると測定誤差の原因となります。
- (5) 温度の安定している場所に設置してください。ロードセルは温度変化に対して敏感であるため、測定誤差の原因となります。
- (6) 温度差のある場所の間で移動したときは、しばらく時間をおいてから測定してください。
- (7) 計量皿に極端に熱いもの、冷たいものを載せないでください。
- (8) 「電子はかり」に衝撃を与えないでください。破損の原因となります。
- (9) 計量皿に 2 kg 以上の荷重を加えないでください。ロードセル破損の原因となります。
- (10) 計量皿にものを載せるときは、静かに載せてください。計量皿に衝撃を与えるとロードセル破損の原因となります。
- (11) 計量皿を引き上げないでください。ロードセル破損の原因となります。
- (12) 電源投入後、測定値が安定するまでにしばらく時間がかかる場合があります。

2.2 修理・改修上の注意事項

- (1) ロードセルのひずみゲージビニル電線は細く切れやすいので、取り扱いには注意して下さい。
- (2) ロードセルに過負荷（2 kg 以上）を与えるとひずみゲージが断線、破損するので取り扱いには注意して下さい。
- (3) ロードセルから計量皿を外さないでください。ロードセルに無理な力が加わり破損する場合があります。
- (4) ロードセルは、ベースに固定していますが、個々の固定具合、またはロードセル個々の特性により、実測値が参考値と異なる場合があります。
- (5) ロードセルのリード線が計量皿に触れないようにしてください。測定誤差の原因になります。
- (6) ICSP 機能を用いてプログラムを書き換えると、EEPROM の内容が書き換わる場合があります。
- (7) 修理・改修完了後に、「電子はかり」の調整、校正を行ってください。

3. 回路の調整

3.1 調整手順

「電子はかり」の調整手順を、以下に示します。調整は校正の前に行ってください。通常、調整は一度行えば、再度調整の必要はありません。

- (1) 計量皿に何も載せない状態、かつスイッチ A、スイッチ B、スイッチ C を押さない状態で電源スイッチをオンにしてください。測定モードになります。
- (2) 半固定抵抗器 VR1、および VR2 を調整し、それぞれチェック端子 TP2、および TP11 の電圧を GND に対して 2.500 V に設定してください。
- (3) LCD 表示器の表示が適切に見えるように、半固定抵抗器 VR51 を調整してください。
- (4) 電源スイッチをオフにしてください。

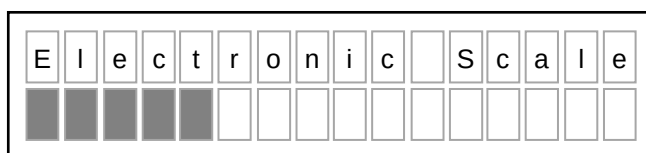
4. 測定モード

4.1 概要

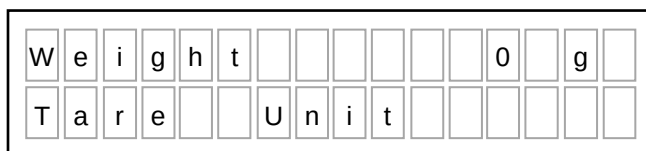
計量皿に載せたものの重さを量ることができます。「電子はかり」の本来の機能です。

4.2 操作方法

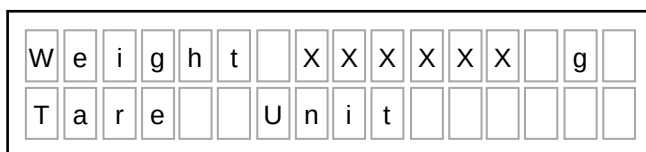
計量皿に何も載せない状態で、スイッチ A、スイッチ B、スイッチ C を押さずに電源スイッチをオンにすると、測定モードになり、LCD 表示器には、以下のような画面が表示されます。下段のバーが 16 個表示されたら測定準備完了です。



その後、LCD 表示器に以下のように表示されます。電源スイッチをオンにしたときの状態を「0 g」と認識しますので、計量皿にものが載っていても表示は「0 g」となります。

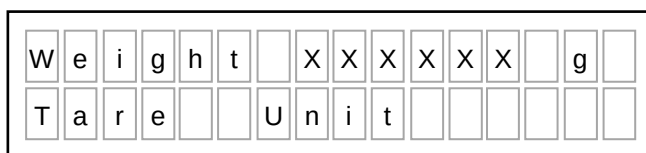


計量皿にものを載せると、その重さを「XXXXXX」で表示します。



スイッチ A [Tare] を押すと、現在の測定値を 0 にします (風袋機能)。風袋機能は、計量皿の上に載せた、容器や袋の重さを除外したり、現在の重さからの増減を測定したりするために用いることができます。

スイッチ B [Unit] を押すと、測定単位を変えることができます。スイッチ B を押すごとに以下のように「g (グラム)」、「lb (ポンド)」、「oz (オンス)」の順で単位が切り替わります。単位は LCD 表示器の 1 行目の 15, 16 桁目に表示されます。なお、 $1000\text{ g} = 2.205\text{ lb} = 35.27\text{ oz}$ の関係があります。



W	e	i	g	h	t		X	X	X	X	X	X		l	b
T	a	r	e			U	n	i	t						

W	e	i	g	h	t		X	X	X	X	X	X		o	z
T	a	r	e			U	n	i	t						

計量皿に載せたものの重さが、およそ 1100 g (2.43 lb, 38.8 oz) を超えると、LCD 表示器に以下のように表示されます。この場合は、測定範囲を超えていますので、計量皿に載せているものを取り除いてください。

また、計量皿をおよそ 100 g 以上の力で引き上げた場合も、破損を防止するため、以下のような表示がされます。

-	-		O	v	e	r		L	o	a	d		-	-	
T	a	r	e			U	n	i	t						.

「測定モード」、および以下に述べる「校正モード」、「動作確認モード」を起動するため以外の操作をした場合（指定されたスイッチ以外を押して電源をオンにした場合）、LCD 表示器に以下のように表示されます。いったん電源スイッチをオフにした後、再度、正しい操作によって、電源スイッチをオンにしてください。

P	l	e	a	s	e		P	o	w	e	r		O	F	F
															.

5. 校正モード

5.1 概要

「電子はかり」を使用する前に、0 kg と 1 kg の基準値（AD 変換値）を記録しておく必要があります。校正モードでは、これらの基準値を測定し、PIC 内部の EEPROM に保存します。

一度、校正を行い、正しい校正値が保存されると、その後は、保存された校正値を用いて、測定を行います。電源を切っても校正値は EEPROM に保存されているので、毎回、校正を行う必要はありません。

5.2 操作方法

スイッチ A だけを押しながら電源スイッチをオンにすると、校正モードとなり、LCD 表示器に以下のような画面が約 1 秒間表示されます。

C	a	l	i	b	r	a	t	i	o	n		M	o	d	e

その後、0 kg 校正モードとなり、LCD 表示器に、以下のように表示されます。表示されている「XXXXXX」は、現在の AD 変換値を表しています。ここで、計量皿に何も載せない状態（0 kg）にしてください。

0	k	g		C	a	l				X	X	X	X	X	X
C	a	l				1	k	g				E	x	i	t

計量皿に何も載せない状態（0 kg）の校正値を EEPROM に保存するためには、スイッチ A [Cal] を押します。EEPROM に保存が完了すると、1 秒間、以下のように表示されます。

E	E	P	R	O	M		W	r	i	t	e		O	K	
C	a	l				1	k	g				E	x	i	t

次に 1 kg の校正を行います。スイッチ B [1kg] を押すと、LCD 表示器には、以下のように表示され、1 kg 校正モードになります。表示されている「XXXXXX」は、現在の AD 変換値を表しています。ここで、計量皿に正確な 1 kg のおもりを計量皿に載せてください。

1	k	g		C	a	l				X	X	X	X	X	X
C	a	l				0	k	g				E	x	i	t

計量皿に正確な 1 kg のおもりを計量皿に載せた状態の校正値を EEPROM に保存するためには、スイッチ A [Cal] を押します。EEPROM に保存が完了すると、1 秒間、以下のように表示されます。

E	E	P	R	O	M		W	r	i	t	e		O	K	
C	a	l				0	k	g				E	x	i	t

再度、0 kg の校正を行う場合は、スイッチ B [0kg] を押すと、0 kg 校正モードになります。また、0 kg 校正モードから 1 kg 校正モードに移るときはスイッチ B [1kg] を押すと 1 kg 校正モードになります。校正モードを変えただけでは、校正値は EEPROM に保存されません。

校正モードを終了するためには、スイッチ C [Exit] を押してください。約 1 秒間、LCD 表示器に以下のように表示された後に、自動的にリセットし、再起動します。

	-	-	-		R	e	s	e	t		-	-	-		

6. 動作確認モード

6.1 概要

「電子はかり」の動作状況を確認するため、AD 変換値、校正データが保存された EEPROM の内容を表示することができます。また、AD 変換器と PIC との間の SPI 通信を連続して行うことができます。

6.2 操作方法

スイッチ B だけを押しながら電源スイッチをオンにすると、動作確認モードとなり、LCD 表示器に以下のようなメニュー画面が表示されます。

			C	h	e	c	k		M	o	d	e			
A	D			E	E	P	R	O	M				S	P	I

(1) AD 変換値確認モード

メニュー画面が表示された状態で、スイッチ A [AD] を押すと、AD 変換値確認モードになり、LCD 表示器に以下のような画面が表示されます。表示されている「XXXXXXX」は、現在の AD 変換値を表しています。

表示されている AD 変換値は、AD 変換を 1000 回行った結果得られた平均値を示しています。

A	D		V	a	l	u	e			X	X	X	X	X	X
M	e	n	u									E	x	i	t

ここで、スイッチ A [Menu] を押すと、メニュー画面に戻ります。

スイッチ C [Exit] を押すと、約 1 秒間、LCD 表示器に以下のように表示された後に、自動的にリセットし、再起動します。

	-	-	-		R	e	s	e	t		-	-	-		

(2) メモリ内容確認モード

メニュー画面が表示された状態で、スイッチ B [EEPROM] を押すと、メモリ内容確認モードになり、LCD 表示器には、以下のような画面が表示されます。

ここで、「AABBCCDDEEFF」は、メモリ内容を 16 進数で表しています。「AA」は 0 番地、「BB」は 1 番地、以下、「FF」は 5 番地の内容です。0 番地と 1 番地には 0 kg の校正值、2 番地と 3 番地には 1 kg の校正值、4 番地と 5 番地には 0 kg と

1 kg との校正值の差を保存しています（下位番地が上位バイト，上位番地が下位バイト）.

R	O	M	:	A	A	B	B	C	C	D	D	E	E	F	F
M	e	n	u									E	x	i	t

ここで，スイッチ A [Menu] を押すと，メニュー画面に戻ります.

スイッチ C [Exit] を押すと，約 1 秒間，LCD 表示器に以下のように表示された後に，自動的にリセットし，再起動します.

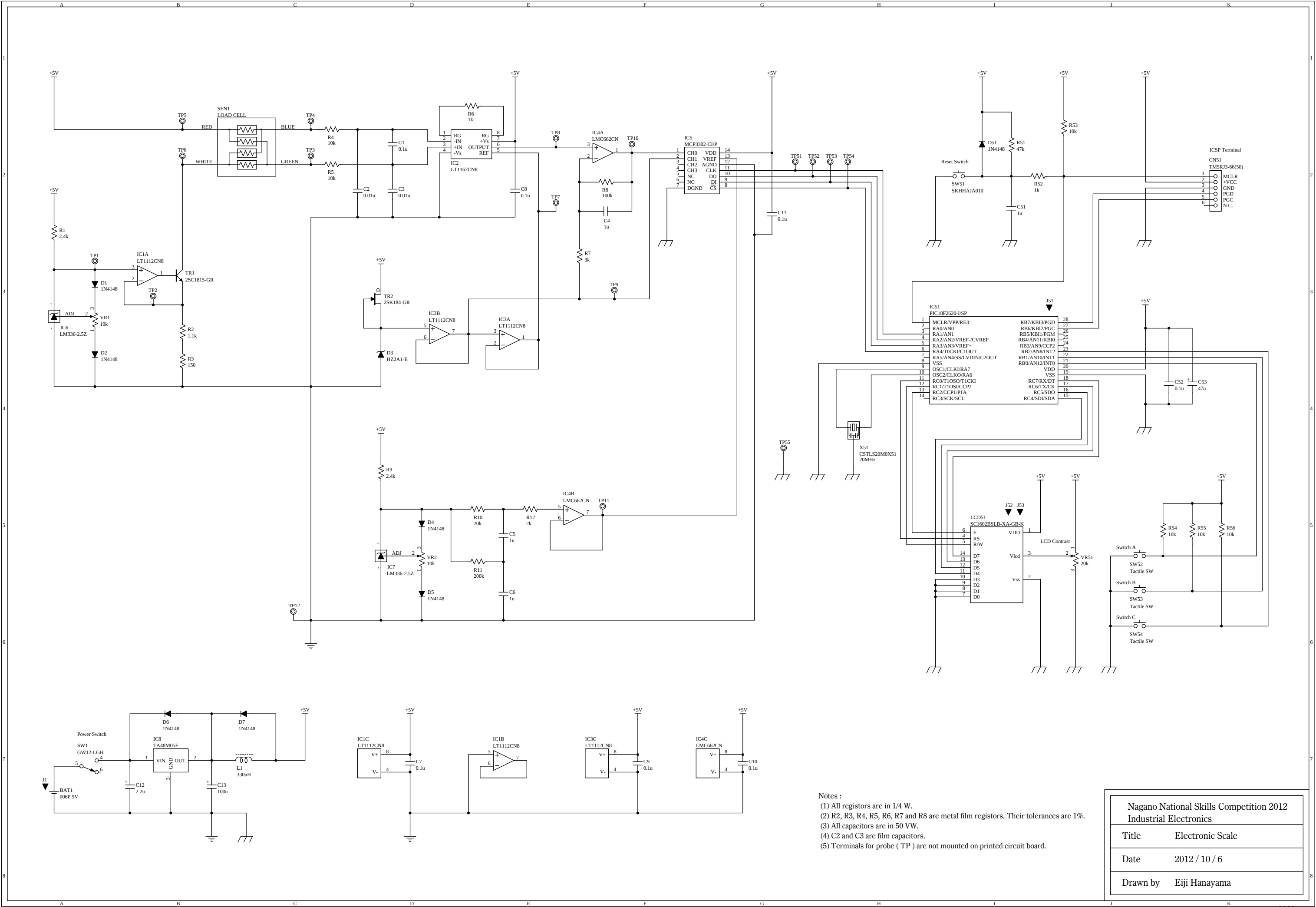
	-	-	-		R	e	s	e	t		-	-	-		

(3) SPI 信号観測モード

メニュー画面が表示された状態で，スイッチ C [SPI] を押すと，SPI 信号観測モードになり，LCD 表示器には，以下のような画面が表示されます. PIC と MCP3302-CI/P との間の SPI 通信が連続して行われます.

S	P	I		M	e	a	s	u	r	e	m	e	n	t	
	R	e	s	e	t		t	o		E	x	i	t		

SPI 信号観測モードを終了するためには，電源スイッチをオフにするか，リセットスイッチを押下してください. このモードからは，メニュー画面に戻ることはできません.



- Notes :
- (1) All registers are in 1/4 W.
 - (2) R2, R3, R4, R5, R6, R7 and R8 are metal film registers. Their tolerances are 1%.
 - (3) All capacitors are in 50 VW.
 - (4) C2 and C3 are film capacitors.
 - (5) Terminals for probe (TP) are not mounted on printed circuit board.

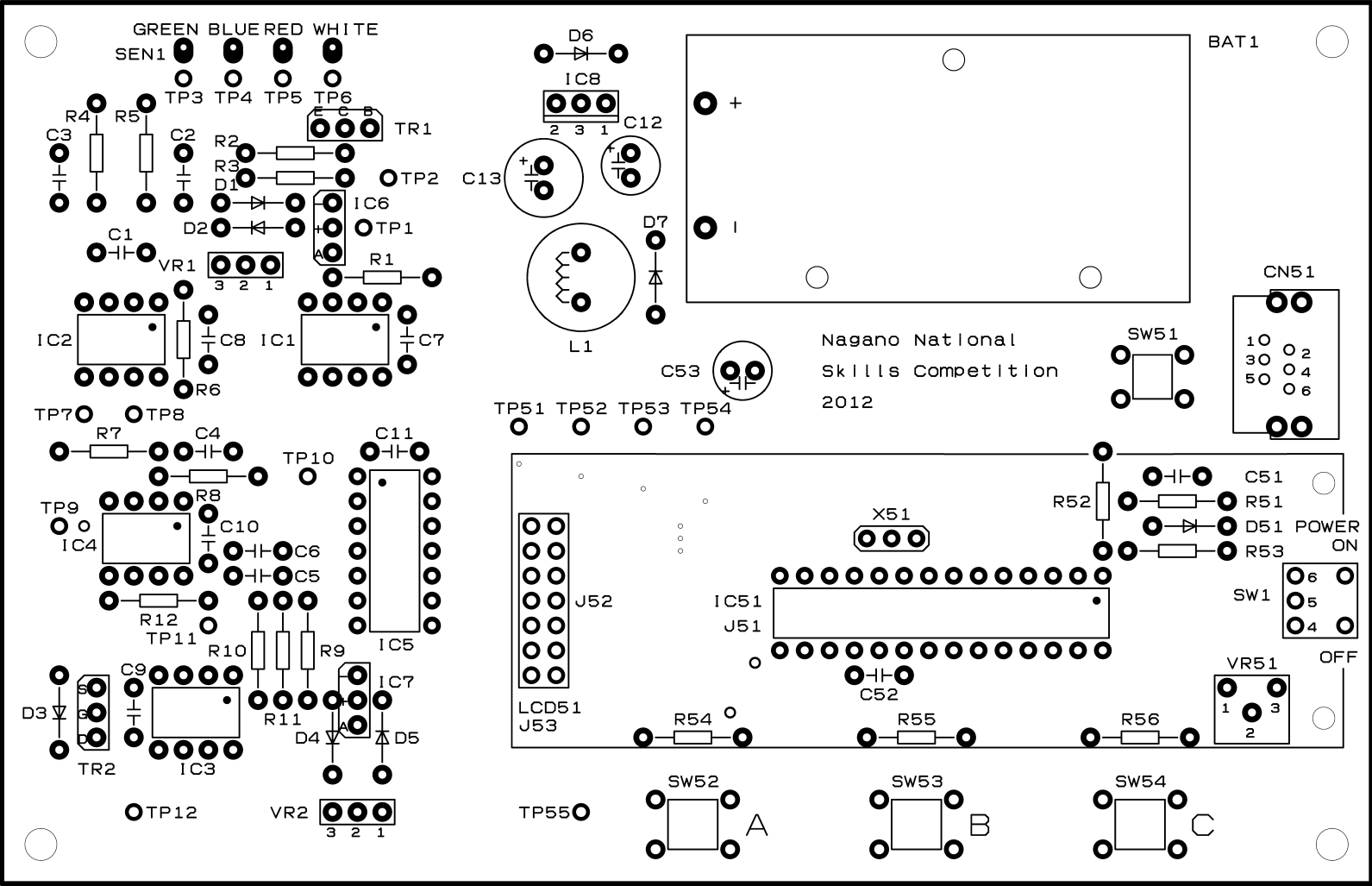
Nagano National Skills Competition 2012 Industrial Electronics	
Title	Electronic Scale
Date	2012 / 10 / 6
Drawn by	Eiji Hanayama

電子はかり 部品表

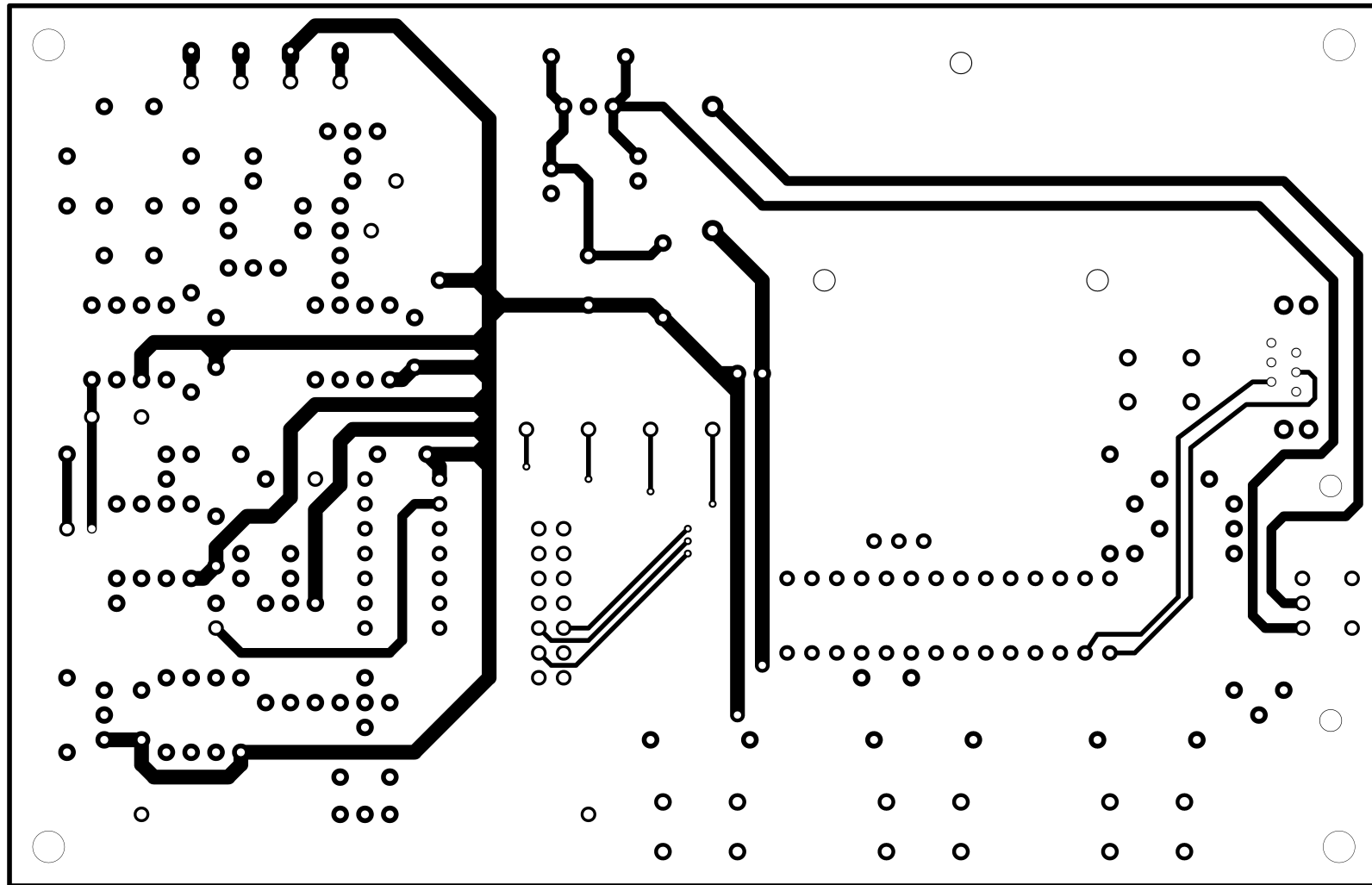
部品記号	品名	形状	定格・形式	製造会社	数量	備考	支給可能部品	データシート
IC1, 3	ピコアンペア入力デュアルオペアンプ	DIP	LT1112CN8	Linear Technology	2		○	○
IC2	高精度計装アンプ	DIP	LT1167CN8	Linear Technology	1		○	○
IC4	CMOSデュアルオペアンプ	DIP	LMC662CN	National Semiconductor	1		○	○
IC5	ADコンバータ	DIP	MCP3302-CI/P	Microchip Technology	1		○	○
IC6, 7	シャントレギュレータ	リード	LM336Z-2.5	National Semiconductor	2		○	○
IC8	低損失三端子レギュレータ	リード	TA48M05F	東芝セミコンダクタ	1		○	○
IC51	PICマイコン	DIP	PIC18F2620-I/SP	Microchip Technology	1		×	○
TR1	トランジスタ	リード	2SC1815-GR	東芝セミコンダクタ	1		○	○
TR2	電界効果トランジスタ	リード	2SK184-GR	東芝セミコンダクタ	1		○	○
D1, 2, 4～7, 51	汎用小信号高速スイッチング・ダイオード	リード	1N4148	Fairchild Semiconductor	7		○	○
D3	定電圧ダイオード	リード	HZ2A1-E	ルネサス	1		○	○
R1, 9	炭素皮膜抵抗器 2.4kΩ±5% 1/4W	リード	CF 1/4C 242J	KOA	2		○	○
R10	炭素皮膜抵抗器 20kΩ±5% 1/4W	リード	CF 1/4C 203J	KOA	1		○	○
R11	炭素皮膜抵抗器 200kΩ±5% 1/4W	リード	CF 1/4C 204J	KOA	1		○	○
R12	炭素皮膜抵抗器 2kΩ±5% 1/4W	リード	CF 1/4C 202J	KOA	1		○	○
R51	炭素皮膜抵抗器 47kΩ±5% 1/4W	リード	CF 1/4C 473J	KOA	1		○	○
R52	炭素皮膜抵抗器 1kΩ±5% 1/4W	リード	CF 1/4C 102J	KOA	1		○	○
R53～56	炭素皮膜抵抗器 10kΩ±5% 1/4W	リード	CF 1/4C 103J	KOA	4		○	○
R2	金属皮膜抵抗器 1.1kΩ±1% 1/4W	リード	MF 1/4CC 1101F	KOA	1		○	○
R3	金属皮膜抵抗器 150Ω±1% 1/4W	リード	MF 1/4CC 1500F	KOA	1		○	○
R4, 5	金属皮膜抵抗器 10kΩ±1% 1/4W	リード	MF 1/4CC 1002F	KOA	2		○	○
R6	金属皮膜抵抗器 1kΩ±1% 1/4W	リード	MF 1/4CC 1001F	KOA	1		○	○
R7	金属皮膜抵抗器 3kΩ±1% 1/4W	リード	MF 1/4CC 3001F	KOA	1		○	○
R8	金属皮膜抵抗器 100kΩ±1% 1/4W	リード	MF 1/4CC 1003F	KOA	1		○	○
VR1, 2	多回転半固定抵抗器(トリマポテンションメータ) 10kΩ 25回転	リード	3296W-1-103	Bourns	2		○	○
VR51	半固定抵抗器(トリマポテンションメータ) 20kΩ	リード	GF063X203	東京コスモス	1		○	○
C1, 7～11, 52	積層セラミックコンデンサ 0.1μF/50V	リード	RPEF11H104Z2K1A01B	村田製作所	7	5.0mmピッチ	○	○
C4～6, 51	積層セラミックコンデンサ 1μF/50V	リード	RDER71H105K2K1C03B	村田製作所	4	5.0mmピッチ	○	○
C2, 3	ポリエステルフィルムコンデンサ 0.01μF/50V	リード	50F2D103J	ルビコン	2	4.0mmピッチ	○	○
C12	電解コンデンサ 2.2μF/50V	リード	ESMG500ELL2R2ME11D	日本ケミコン	1		○	○
C13	電解コンデンサ 100μF/50V	リード	ECA1HM101	Panasonic	1	φ8mm	○	○
C53	電解コンデンサ 47μF/50V	リード	ESMG500ELL470MF11D	日本ケミコン	1		○	○
L1	電源回路用インダクタ 330μH	リード	TSLI112RA-331KR82-PF	TDK	1		○	○
X51	セラミック発振子 コンデンサ内蔵タイプ 20MHz	リード	CSTLS20M0X51	村田製作所	1		×	○
SEN1	ロードセル 1mV/V 2kgF.S. R=1kΩ	ビニル電線	デジタルクッキングスケール KD-189-WH	タニタ	1	取外し品	×	×
LCD51	LCDキャラクタディスプレイモジュール 16×2行 バックライト付	モジュール	SC1602BSLB-XA-GB-K	Sunlike Display Tech.	1		×	○
SW1	パドルスイッチ GW型 PC-H端子型 単極双投 灰色	ピン	GW-12LGH	日本開閉器	1		×	○
SW51	タクトスイッチ 6mm角	ピン	SKHHAJA010	アルプス電気	1		×	○
SW52	タクトスイッチ (赤色)	ピン	通販コード P-03646	秋月電子通商	1		×	○
SW53	タクトスイッチ (黄色)	ピン	通販コード P-03650	秋月電子通商	1		×	○

電子はかり 部品表

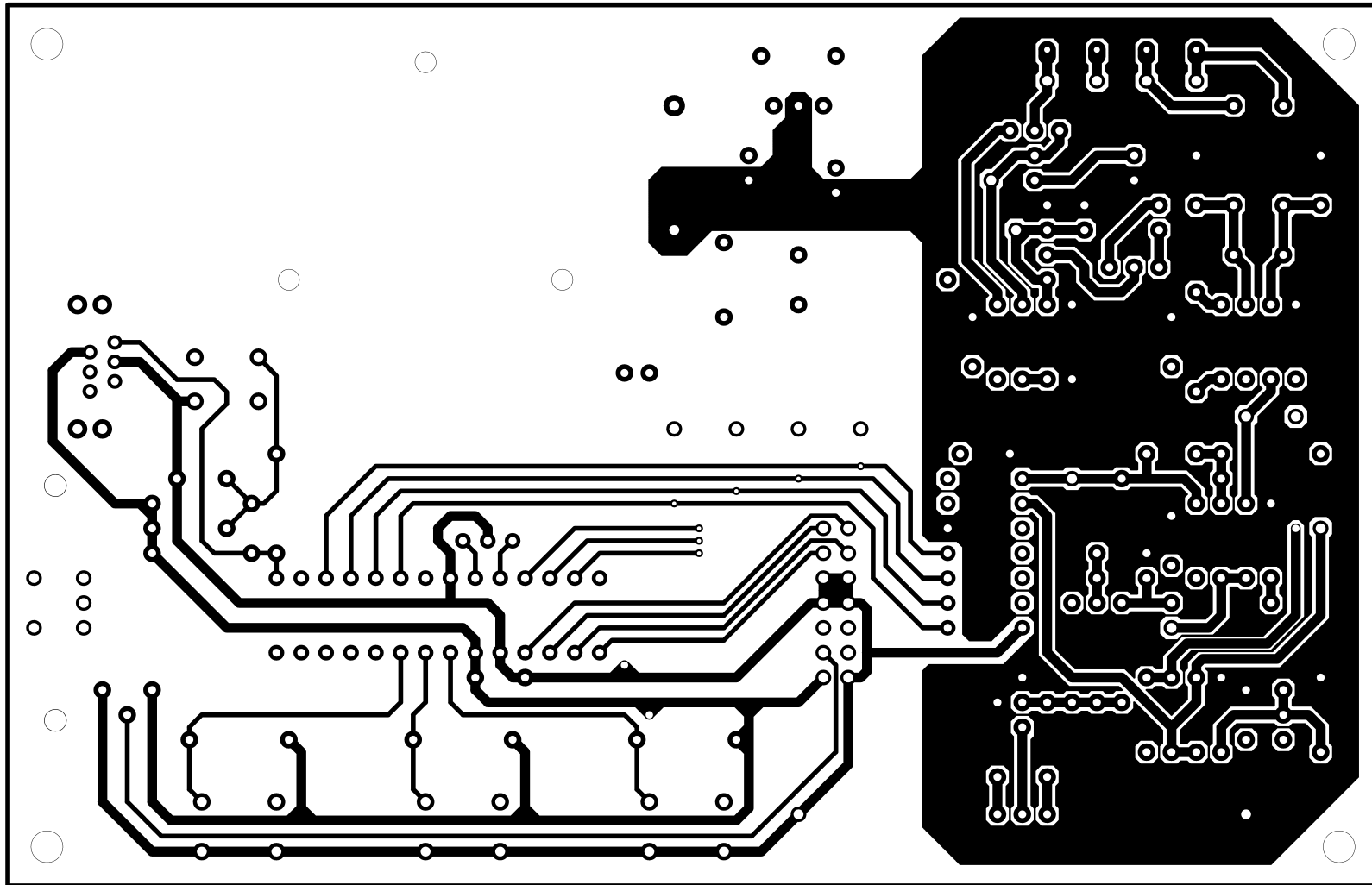
部品記号	品名	形状	定格・形式	製造会社	数量	備考	支給可能部品	データシート
SW54	タクトスイッチ（緑色）	ピン	通販コード P-03651	秋月電子通商	1		×	○
CN51	モジュラジャック ライトアングルディップ 6極	ピン	TM5RJ3-66(50)	ヒロセ電機	1	ICSP用	×	○
J1	006P電池ホルダ	ピン	BH-9VPC	Confortable Electoronic	1	タカチ BH-9V-3P 相当品	×	○
J51	ICソケット 28ピン	DIP	2227-28-03	Neltron Industrial	1	相当品可	×	○
J52	ピンソケット メス 2×7 (14P)	DIP	通販コード P-00169	秋月電子通商	1	LCDモジュール用(基板側)	×	○
J53	ピンヘッダ オス 2×7 (14P)	DIP	通販コード P-00166	秋月電子通商	1	LCDモジュール用(LCD側)	×	○
TP**	オシロブローブ用チェック端子	ピン	LC-2-S-黒	マックエイト	2	必要ならば各自実装	○	○
TP**	オシロブローブ用チェック端子	ピン	LC-2-S-赤	マックエイト	1	必要ならば各自実装	○	○
TP**	オシロブローブ用チェック端子	ピン	LC-2-S-黄	マックエイト	4	必要ならば各自実装	○	○
TP**	オシロブローブ用チェック端子	ピン	LC-2-S-白	マックエイト	4	必要ならば各自実装	○	○
BAT1	006P型アルカリ乾電池 9V			(メーカー不問)	1		×	×
PB1	電子はかり専用基板			P板.com	1		×	×
シャーシ類	アルミニウムベース			特注品	1		×	×
	ロードセルスペーサ			特注品	1		×	×
	計量皿		デジタルクッキングスケール KD-189-WH	タニタ	1	取外し品	×	×
	ジョイント		デジタルクッキングスケール KD-189-WH	タニタ	1	取外し品	×	×
ねじ類	黄銅 ナベ小ねじ M2 L=8mm		B-0208	廣杉計器	3	電池ホルダ取付用	×	×
	黄銅 六角ナット(1極) M2		BNT-02	廣杉計器	3	電池ホルダ取付用	×	×
	リン青銅 スプリングワッシャ M2 φ 4.4mm		BSW-02	廣杉計器	3	電池ホルダ取付用	×	×
	平座金 M2 外径 φ 4.0mm-内径 φ 2.2mm		BW-0243-03	廣杉計器	3	電池ホルダ取付用	×	×
	ポリカーボネイト セットナベ小ねじ M2 L=5mm		PC-0205-T	廣杉計器	4	LCDモジュール取付用	×	×
	ジュラコンスペーサ M2 L=11mm		AS-2011	廣杉計器	2	LCDモジュール取付用	×	×
	黄銅セットナベ小ねじ(SW, PW付) M3 L=8mm		B-0308-S1	廣杉計器	6	ベース, 基板取付用	×	×
	黄銅スペーサ 丸型オネジメネジ(スリ割付) M3 L=5mm		BRB-305SE	廣杉計器	4	ベース用スペーサ(手前, 中側)	×	×
	黄銅スペーサ 丸型両メネジ(スリ割付) M3 L=5mm		ARB-305SE	廣杉計器	2	ベース用スペーサ(奥側)	×	×
	黄銅スペーサ 丸型両メネジ(スリ割付) M3 L=10mm		ARB-310SE	廣杉計器	4	基板取付用スペーサ	×	×
	2.5mm六角穴付きねじ M4 L=10mm		デジタルクッキングスケール KD-189-WH	タニタ	4	ロードセル取付用(取外し品)	×	×
おもり	ペットボトル（重さ500g, 水入り）				2		×	×
	ペットボトル（重さ不明, ビー玉入り）				1		×	×
修理用材料	炭素皮膜抵抗器 各種 ±5% 1/4W	リード	CF 1/4C ***J	KOA			○	○
	ジュンフロンテフロンETFE電線		0.26mm φ, 各色	潤工社			○	○
	はんだ		共晶はんだ	各社			○	×



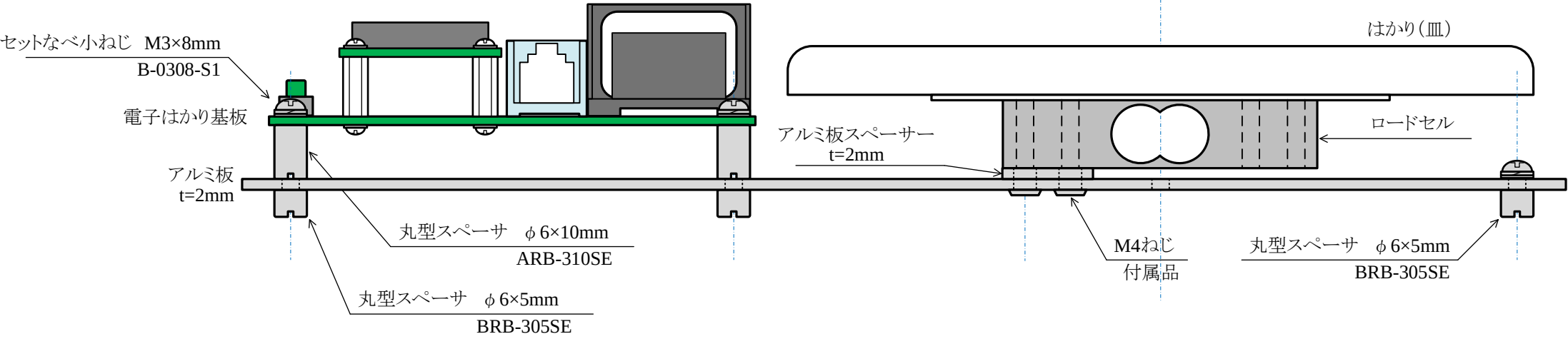
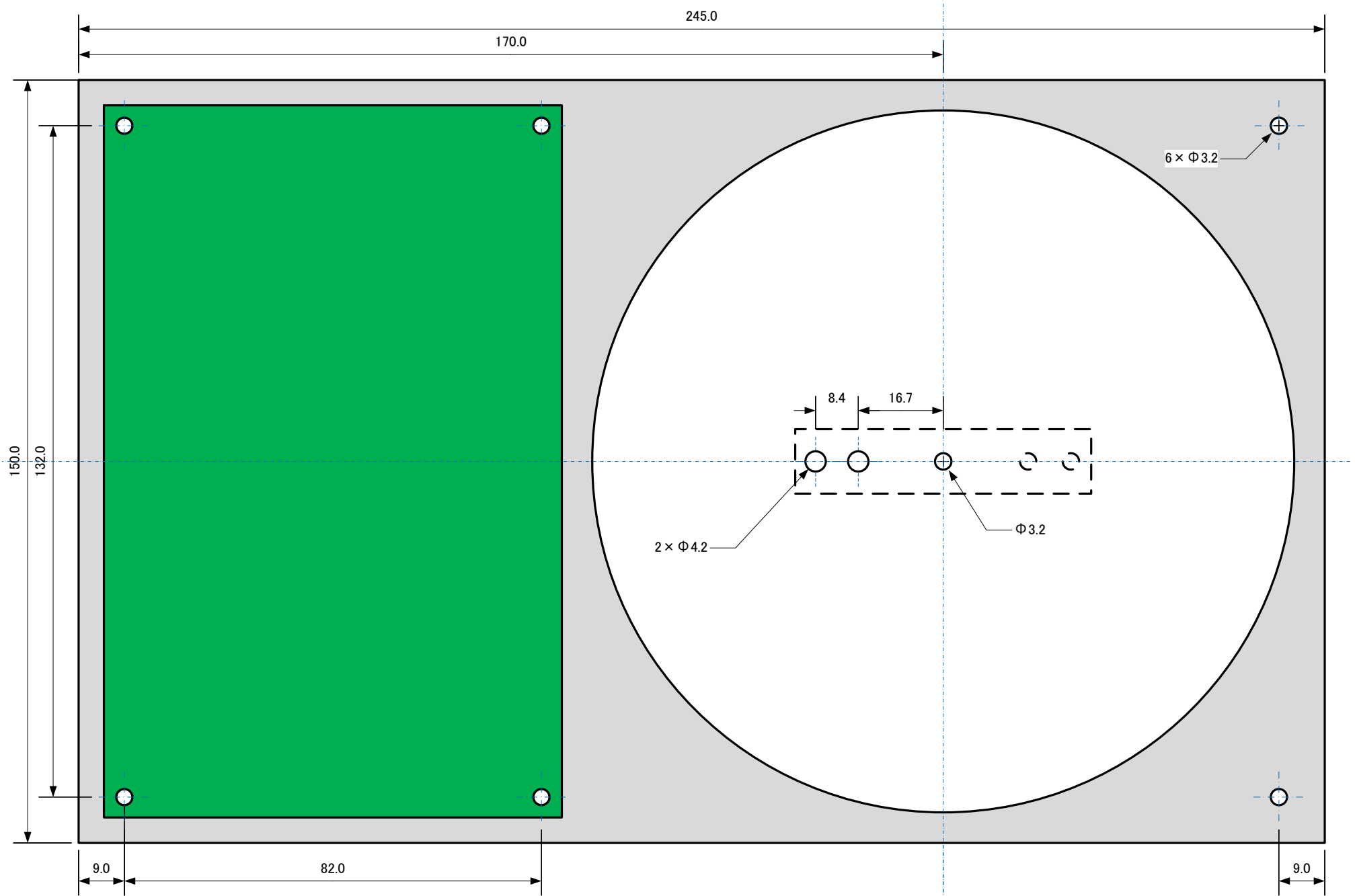
「電子はかり」 基板部品配置図



「電子はかり」 基板部品面配線図



「電子はかり」 基板はんだ面配線図



電子はかり組立図	1:1 (A3)
2012.10.19	y·yajima

C 言語プログラムソースリスト(hakari_00name.c)

```

1  /*****
2  * 第50回技能五輪全国大会「電子機器組立て」職種
3  * 競技Ⅱ 2012/10/28 実施
4  * 「電子はかり」プログラム
5  * File name      : hakari_00name.c
6  * CPU            : Microchip PIC18F2620-I/SP
7  * Clock          : 20MHz (Ceramic Resonator)
8  * Date           : 2012/10/20
9  * Programmed by  Eiji Hanayama
10 *****/
11
12 /* ***** インクルードファイル ***** */
13
14 #include <p18f2620.h>
15 #include <delays.h>
16 #define CLOCK_FREQ 20000000
17 #include "../lcdlib_c18_v032.h"
18
19 /* ***** コンフィギュレーションの設定 ***** */
20
21 #pragma config OSC=HS, FCMEN=ON, IESO=OFF //CONFIG1H
22 #pragma config PWRT=ON, BOREN=SBORDIS, BORV=3 //CONFIG2L
23 #pragma config WDT=OFF, WDTPS=1 //CONFIG2H
24 #pragma config MCLRE=ON, LPT1OSC=OFF, PBAEN=OFF, CCP2MX=PORTC //CONFIG3H
25 #pragma config STVREN=ON, LVP=OFF, XINST=OFF, DEBUG=OFF //CONFIG4L
26 #pragma config CP0=OFF, CP1=OFF, CP2=OFF, CP3=OFF //CONFIG5L
27 #pragma config CPB=OFF, CPD=OFF //CONFIG5H
28 #pragma config WRT0=OFF, WRT1=OFF, WRT2=OFF, WRT3=OFF //CONFIG6L
29 #pragma config WRTB=OFF, WRTC=OFF, WRTD=OFF //CONFIG6H
30 #pragma config EBTR0=OFF, EBTR1=OFF, EBTR2=OFF, EBTR3=OFF //CONFIG7L
31 #pragma config EBTRB=OFF //CONFIG7H
32
33 /* ***** 定義 ***** */
34
35 #define SCLK LATAbits.LATA1 // SPI 通信 Clock
36 #define DOUT PORTAbits.RA2 // SPI 通信 Data Output
37 #define DIN LATAbits.LATA3 // SPI 通信 Data Input
38 #define CS LATAbits.LATA4 // SPI 通信 Chip Select
39 #define SW_A PORTBbits.RB0 // 入力スイッチ A
40 #define SW_B PORTBbits.RB1 // 入力スイッチ B
41 #define SW_C PORTBbits.RB2 // 入力スイッチ C
42
43 /* ***** プロトタイプ宣言 ***** */
44
45 void scale_mode(void);
46 void calibration_mode(void);
47 void check_mode(void);
48 void display_ad_value(void);
49 void display_eeprom(void);
50 void output_spi(void);
51 void initialize(void);
52 void reset(void);
53 void switch_status_set(void);
54 void switch_check(void);
55 float abs(float number);
56 float power10(int number);
57 unsigned char number_length(int number);
58 int ad_conv(void);
59 int get_MCP3302(void);
60 void spi_clock(void);
61 int read_int_EEPROM(unsigned char address);
62 void write_int_EEPROM(unsigned char address, int data);
63 unsigned char read_EEPROM(unsigned char address);
64 void write_EEPROM(unsigned char address, unsigned char data);
65
66 /* ***** グローバル変数宣言 ***** */
67
68 int zero_value; // 0kg のときの AD 変換値

```

```

69 int onekg_value;    // 1kg のときの AD 変換値
70 int zero_one;      // 0kg と 1kg との AD 変換値の差
71 unsigned char switch_status[3] = {0, 0, 0}; // 現在のスイッチの状態変数
72
73 /* ***** メイン関数 ***** */
74
75 void main(void)
76 {
77     initialize();    // 初期化
78
79     // スイッチを何も押さずに起動
80     if ((SW_A == 1) && (SW_B == 1) && (SW_C == 1))
81     {
82         scale_mode();    // 測定モード
83     }
84     // スイッチ A だけを押し起動
85     else if ((SW_A == 0) && (SW_B == 1) && (SW_C == 1))
86     {
87         calibration_mode();    // 校正モード
88     }
89     // スイッチ B だけを押し起動
90     else if ((SW_A == 1) && (SW_B == 0) && (SW_C == 1))
91     {
92         check_mode();    // 動作確認モード
93     }
94     // 上記以外の場合
95     else
96     {
97         LCD_String("Please Power OFF");    // 電源のオフを促すメッセージ表示
98         while (1)
99         {
100             }
101     }
102 }
103
104 /* ***** 関数 ***** */
105
106 /** 測定モード **/
107 void scale_mode(void)
108 {
109     unsigned char i;    // ループ制御変数
110     int ad_value;    // AD 変換値の格納用変数
111     float weight;    // 測定重量の格納用変数
112     unsigned char unit = 0;    // 単位を表す変数
113     float unit_multi[3] = {1000.0, 2.205, 35.27};    // 単位変換係数
114     char unit_str[7] = "g lboz";    // 各単位の記号
115     unsigned char unit_keta1[3] = {6, 2, 3};    // 各単位の整数部の表示桁数
116     unsigned char unit_keta2[3] = {0, 3, 2};    // 各単位の小数部の表示桁数
117     int over_load_U;    // 過負荷値 (重量超過)
118     int over_load_D;    // 過負荷値 (計量皿引っ張り)
119
120     // 測定モードタイトル表示
121     LCD_String("Electronic Scale");
122     for (i = 0; i < 16; i++)    // ウェイト表示 (キャパシタ充電時間確保)
123     {
124         LCD_Locate(1, i);
125         LCD_Character(0xFF);
126         Delay10KTCYx(100);    // 0.2 秒時間待ち
127     }
128     // 測定モードメニュー表示
129     LCD_Clear();
130     LCD_Locate(1, 0);
131     LCD_String("Tare Unit    ");
132
133     // EEPROM からのデータ読み出し
134     zero_value = read_int_EEPROM(0);
135     onekg_value = read_int_EEPROM(2);
136     zero_one = read_int_EEPROM(4);
137     // 過負荷値の設定
138     over_load_U = (float)onekg_value + (float)zero_one * 0.1;
139     over_load_D = (float)zero_value - (float)zero_one * 0.1;

```

```

140 // 電源をオンにしたときのAD変換値を0kgとする
141 zero_value = ad_conv();
142
143 // 重量測定
144 while (1)
145 {
146     // AD変換値の取り込み
147     ad_value = ad_conv();
148     // AD変換値を重さに変換
149     weight = (((float)(ad_value - zero_value)) / (float)zero_one) * unit_multi[unit];
150
151     LCD_Home();
152     // 重量超過, 計量皿引上げの表示
153     if ((ad_value > over_load_U) || (ad_value < over_load_D))
154     {
155         LCD_String("-- Over Load -- ");
156     }
157     // 測定結果の表示
158     else
159     {
160         LCD_String("Weight          ");
161         LCD_Locate(0, 7);
162         // -0を0と表示するための処理
163         if (abs(weight) < 5.0 / power10(unit_keta2[unit]+1))
164         {
165             weight = 0.0;
166         }
167         LCD_FNumber(weight, unit_keta1[unit], unit_keta2[unit]); // 測定値の表示
168         LCD_Locate(0, 14);
169         for (i = unit*2; i < unit*2+2; i++)
170         {
171             LCD_Character(unit_str[i]); // 単位の表示
172         }
173     }
174
175     // スイッチAが押された場合
176     if ((SW_A == 0) && (switch_status[0] == 0)) // 風袋機能
177     {
178         ad_value = zero_value;
179         switch_status[0] = 1;
180     }
181     // スイッチBが押された場合
182     else if ((SW_B == 0) && (switch_status[1] == 0)) // 単位変換
183     {
184         unit = (unit + 1) % 3;
185         switch_status[1] = 1;
186     }
187     // 上記以外の場合
188     else
189     {
190         // なにもしない
191     }
192     switch_check();
193 }
194
195
196
197 /*** 校正モード ***/
198 void calibration_mode(void)
199 {
200     unsigned char i; // ループ制御変数
201     int cal_value; // 校正値
202     unsigned char length; // 値の桁数を表す変数
203     unsigned char calibration_weight = 0; // 校正を行う重さを表す変数
204     unsigned char disp = 0; // LCDの表示の制御用変数
205
206     switch_status_set(); // スイッチの状態変数設定
207
208     // 校正モードタイトル表示
209     LCD_Clear();
210     LCD_String("Calibration Mode");
211     Delay10KTCYx(250); // 1秒時間待ち

```

```

212     Delay10KTCYx(250);
213
214     while (1)
215     {
216         // 校正モードメニュー表示
217         if (disp == 0)
218         {
219             LCD_Home();
220             LCD_Number(calibration_weight);
221             LCD_String("kg Cal      ");
222             LCD_Locate(1, 0);
223             LCD_String("Cal  ");
224             LCD_Number(!calibration_weight & 1);
225             LCD_String("kg  ");
226             LCD_String("Exit");
227             disp = 1;
228         }
229         // AD 変換値の取り込み
230         cal_value = ad_conv();
231         // AD 変換値の表示
232         LCD_Locate(0, 10);
233         LCD_Number(cal_value);
234         length = number_length(cal_value);
235         for (i = 10+length; i < 16; i++)
236         {
237             LCD_Character(0x20);
238         }
239         // スイッチ A だけが押された場合
240         if ((SW_A == 0) && (SW_B == 1) && (SW_C == 1) && (switch_status[0] == 0))
241         {
242             // 現在、校正している重さの校正値の書き込み
243             write_int_EEPROM(calibration_weight*2, cal_value);
244             // 1kg と 0kg との差の校正値の再書き込み
245             zero_value = read_int_EEPROM(0);
246             onekg_value = read_int_EEPROM(2);
247             zero_one = onekg_value - zero_value;
248             write_int_EEPROM(4, zero_one);
249             // EEPROM 書き込み完了表示
250             LCD_Home();
251             LCD_String("Write EEPROM OK");
252             Delay10KTCYx(250);           // 1秒時間待ち
253             Delay10KTCYx(250);
254             switch_status[0] = 1;
255             disp = 0;
256         }
257         // スイッチ B だけが押された場合
258         else if ((SW_A == 1) && (SW_B == 0) && (SW_C == 1) && (switch_status[1] == 0))
259         {
260             calibration_weight = !calibration_weight & 1;    // 校正する重さの変更
261             switch_status[1] = 1;
262             disp = 0;
263         }
264         // スイッチ C だけが押された場合
265         else if ((SW_A == 1) && (SW_B == 1) && (SW_C == 0) && (switch_status[2] == 0))
266         {
267             reset();      // リセット
268         }
269         // 上記以外の場合
270         else
271         {
272             // なにもしない
273         }
274         switch_check();
275     }
276 }
277
278
279 /*** 動作確認モード ***/
280 void check_mode(void)
281 {
282     unsigned char disp = 0;    // LCD の表示の制御用変数
283
284     switch_status_set();       // スイッチの状態変数設定

```

```

285     LCD_Clear();
286
287     while (1)
288     {
289         // 動作確認モードのメニュー表示
290         if (disp == 0)
291         {
292             LCD_Home();
293             LCD_String("  Check Mode  ");
294             LCD_Locate(1, 0);
295             LCD_String("AD  EEPROM  SPI");
296             disp = 1;
297         }
298         // スイッチ A だけが押された場合
299         if ((SW_A == 0) && (SW_B == 1) && (SW_C == 1) && (switch_status[0] == 0))
300         {
301             display_ad_value();          // AD 変換値確認モード
302             switch_status[0] = 1;
303             disp = 0;
304         }
305         // スイッチ B だけが押された場合
306         else if ((SW_A == 1) && (SW_B == 0) && (SW_C == 1) && (switch_status[1] == 0))
307         {
308             display_eeprom();            // メモリ内容確認モード
309             switch_status[1] = 1;
310             disp = 0;
311         }
312         // スイッチ C だけが押された場合
313         else if ((SW_A == 1) && (SW_B == 1) && (SW_C == 0) && (switch_status[2] == 0))
314         {
315             output_spi();                // SPI 信号観測モード
316         }
317         // 上記以外の場合
318         else
319         {
320             // なにもしない
321         }
322         switch_check();                  // スイッチのチャタリング、複数回押下判断の防止
323     }
324 }
325
326
327 /*** AD 変換値確認モード ***/
328 void display_ad_value(void)
329 {
330     unsigned char i;                    // ループ制御変数
331     int ad_value;                       // AD 変換値の格納用変数
332     unsigned char length;               // AD 変換値の桁数の格納用変数
333     unsigned char menu = 1;            // メニューに戻るための変数
334
335     switch_status_set();                // スイッチの状態変数設定
336
337     // AD 変換値確認モードメニューの表示
338     LCD_Clear();
339     LCD_String("AD Value");
340     LCD_Locate(1, 0);
341     LCD_String("Menu      Exit");
342
343     while (menu)
344     {
345         // AD 変換
346         ad_value = ad_conv();
347         // AD 変換値の表示
348         LCD_Locate(0, 10);
349         LCD_Number(ad_value);
350         length = number_length(ad_value);
351         for (i = 10+length; i < 16; i++)
352         {
353             LCD_Character(0x20);
354         }
355         // スイッチ A だけが押された場合
356         if ((SW_A == 0) && (SW_B == 1) && (SW_C == 1) && (switch_status[0] == 0))
357         {

```

```

358         menu = 0; // メニューに戻る
359         switch_status[0] = 1;
360     }
361     // スイッチ C だけが押された場合
362     else if ((SW_A == 1) && (SW_B == 1) && (SW_C == 0) && (switch_status[2] == 0))
363     {
364         reset(); // リセット
365     }
366     // 上記以外の場合
367     else
368     {
369         // なにもしない
370     }
371     switch_check(); // スイッチのチャタリング，複数回押下判断の防止
372 }
373 }
374
375
376 /*** メモリ内容確認モード ***/
377 void display_eeprom(void)
378 {
379     unsigned char i; // ループ制御変数
380     unsigned char data; // EEPROM 読み出しデータの格納用変数
381     unsigned char menu = 1; // メニューに戻るための変数
382
383     switch_status_set(); // スイッチの状態変数設定
384
385     // メモリ内容確認モードメニューの表示
386     LCD_Clear();
387     LCD_String("ROM:");
388     LCD_Locate(1, 0);
389     LCD_String("Menu      Exit");
390
391     // EEPROM の 0 番地から 5 番地までの内容を読み出す
392     for (i = 0; i < 6; i++)
393     {
394         data = read_EEPROM(i); // メモリの読み出し
395         LCD_Locate(0, 2*i+4); // メモリ内容の表示
396         LCD_HNumber(data, 2);
397     }
398     while (menu)
399     {
400         // スイッチ A だけが押された場合
401         if ((SW_A == 0) && (SW_B == 1) && (SW_C == 1) && (switch_status[0] == 0))
402         {
403             menu = 0; // メニューに戻る
404             switch_status[0] = 1;
405         }
406         // スイッチ C だけが押された場合
407         else if ((SW_A == 1) && (SW_B == 1) && (SW_C == 0) && (switch_status[2] == 0))
408         {
409             reset(); // リセット
410         }
411         // 上記以外の場合
412         else
413         {
414             // なにもしない
415         }
416         switch_check(); // スイッチのチャタリング，複数回押下判断の防止
417     }
418 }
419
420
421 /*** SPI 信号観測モード ***/
422 void output_spi(void)
423 {
424     int dummy;
425
426     // SPI 信号観測モードメニューの表示
427     LCD_Clear();
428     LCD_String("SPI Measurement ");
429     LCD_Locate(1, 0);

```



```

430     LCD_String(" Reset to Exit ");
431
432     // リセットスイッチが押されるまで SPI 信号を連続して出力
433     while (1)
434     {
435         dummy = get_MCP3302();
436     }
437 }
438
439
440 /** 入出力ポート, LCD の初期化 **/
441 void initialize(void)
442 {
443     // 入出力ポートの初期設定
444     ADCON1 = 0b00001111; // すべてのポートをディジタル入出力に設定
445     TRISA = 0b11100100; // Port A RA1, RA3, RA4 を出力モード, その他を入力モードに設定
446     TRISB = 0b11111111; // Port B すべてのを入力モードに設定
447     // LCD の初期設定
448     LCD_Init();
449     LCD_Clear();
450     LCD_CursorOff();
451 }
452
453
454 /** ソフトウェアリセット **/
455 void reset(void)
456 {
457     // リセットの表示
458     LCD_Clear();
459     LCD_String(" --- Reset --- ");
460     Delay10KTCYx(250); // 1 秒時間待ち
461     Delay10KTCYx(250);
462     // リセット実行 (インラインアセンブラ)
463     {
464         _asm
465             reset // ソフトウェアリセット
466         _endasm
467     }
468 }
469
470
471 /** スwitchの状態変数のセット **/
472 void switch_status_set(void)
473 {
474     switch_status[0] = 1;
475     switch_status[1] = 1;
476     switch_status[2] = 1;
477 }
478
479
480 /** スwitchのチャタリング, 複数回押下判断の防止 **/
481 void switch_check(void)
482 {
483     if ((SW_A == 1) && (switch_status[0] == 1))
484     {
485         Delay10KTCYx(1); // 2ms 時間待ち
486         if (SW_A == 1)
487         {
488             switch_status[0] = 0;
489         }
490     }
491     if ((SW_B == 1) && (switch_status[1] == 1))
492     {
493         Delay10KTCYx(1); // 2ms 時間待ち
494         if (SW_B == 1)
495         {
496             switch_status[1] = 0;
497         }
498     }
499     if ((SW_C == 1) && (switch_status[2] == 1))
500     {
501         Delay10KTCYx(1); // 2ms 時間待ち
502         if (SW_C == 1)

```

```

503         {
504             switch_status[2] = 0;
505         }
506     }
507 }
508
509
510 /** 絶対値を求める **/
511 float abs(float number)
512 {
513     if (number < 0.0)
514     {
515         number = -number;
516     }
517     return(number);
518 }
519
520
521 /** 10 の n 乗 (n >= 1) を求める **/
522 float power10(int number)
523 {
524     int i;                                // ループ制御変数
525     float power = 1.0;                    // 10 の n 乗の計算結果格納
526
527     for (i = 0; i < number; i++)
528     {
529         power *= 10.0;
530     }
531     return(power);
532 }
533
534
535 /** マイナス記号を含めた int 型変数の桁数を求める **/
536 unsigned char number_length(int number)
537 {
538     unsigned char length;                // 桁数の格納用変数
539
540     if (number <= -10000)
541     {
542         length = 6;
543     }
544     else if ((number > -10000) && (number <= -1000))
545     {
546         length = 5;
547     }
548     else if ((number > -1000) && (number <= -100))
549     {
550         length = 4;
551     }
552     else if ((number > -100) && (number <= -10))
553     {
554         length = 3;
555     }
556     else if ((number > -10) && (number <= -1))
557     {
558         length = 2;
559     }
560     else if ((number > -1) && (number < 10))
561     {
562         length = 1;
563     }
564     else if ((number >= 10) && (number < 100))
565     {
566         length = 2;
567     }
568     else if ((number >= 100) && (number <= 1000))
569     {
570         length = 3;
571     }
572     else if ((number >= 1000) && (number <= 10000))
573     {
574         length = 4;
575     }
576     else if (number >= 10000)
577     {

```

```

578         length = 5;
579     }
580     else
581     {
582         // ここには分岐しない
583     }
584     return(length);
585 }
586
587
588 /** AD変換の実行（1000回の平均値をとった値を返す） */
589 int ad_conv(void)
590 {
591     int i;                // ループ制御変数
592     int ad_value;         // AD変換値の格納用変数
593     long ad_sum = 0;      // AD変換値の和の格納用変数
594
595     // 1000回のAD変換値の和を求める
596     for (i = 0; i < 1000; i++)
597     {
598         ad_value = get_MCP3302();
599         ad_sum += ad_value;
600     }
601     // 1000回のAD変換の平均値を求める
602     ad_value = ad_sum / i;
603     return(ad_value);
604 }
605
606
607 /** AD変換器（MCP3302-CI/P）からSPI通信でデータを取り込み */
608 int get_MCP3302(void)
609 {
610     unsigned char i;      // ループ制御変数
611     int Dbuf = 0;         // 取り込みデータの格納用変数
612
613     // スタートビット
614     CS = 0;
615     DIN = 1;
616     spi_clock();
617     // シングルモード(DIN=1), 差動モード(DIN=0)の選択
618     DIN = 0;
619     spi_clock();
620     // チャンネル選択
621     // DIN=X00: CH0(Single), CH0+ CH1-(Differential)
622     // DIN=X01: CH1(Single), CH0- CH1+(Differential)
623     // DIN=X10: CH2(Single), CH2+ CH1-(Differential)
624     // DIN=X11: CH3(Single), CH2- CH1+(Differential)
625     DIN = 0;              // Dummy
626     spi_clock();
627     DIN = 0;              // チャンネル選択上位ビット
628     spi_clock();
629     DIN = 0;              // チャンネル選択下位ビット
630     spi_clock();
631     // サンプル&ホールド完了
632     DIN = 1;              // Dummy
633     spi_clock();
634     DIN = 0;
635     // データをDbufに取り込む
636     for (i = 0; i < 14; ++i)
637     {
638         Dbuf <<= 1;
639         SCLK = 1;         // クロックをHレベルにする
640         if (DOUT == 1)
641         {
642             Dbuf++;
643         }
644         SCLK = 0;         // クロックをLレベルにする
645     }
646     CS = 1;
647     // 13ビットの2の補数を16ビットの2の補数に変換する
648     if ((Dbuf & 0x1000) != 0)
649     {
650         Dbuf = Dbuf | 0xF000;

```

```

651     }
652     return(Dbuf);
653 }
654
655
656 /*** SPI クロック信号の出力 ***/
657 void spi_clock(void)
658 {
659     SCLK = 1;           // クロックを H レベルにする
660     Delay1TCY();        // H レベルで 400ns 時間待ち (Clock 20MHz)
661     Delay1TCY();
662     SCLK = 0;           // クロックを L レベルにする
663     Delay1TCY();        // L レベルで 400ns 時間待ち (Clock 20MHz)
664     Delay1TCY();
665 }
666
667
668 /*** EEPROM からの int 型データの読み出し ***/
669 // (上位バイト: address, 下位バイト: address+1)
670 int read_int_EEPROM(unsigned char address)
671 {
672     int data;           // 読み出しデータの格納用変数
673
674     data = read_EEPROM(address);           // 上位バイトの読み出し
675     data <= 8;
676     data = data + read_EEPROM(address+1); // 下位バイトの読み出し
677     return(data);
678 }
679
680
681 /*** EEPROM への int 型データの書き込み ***/
682 // (上位バイト: address, 下位バイト: address+1)
683 void write_int_EEPROM(unsigned char address, int data)
684 {
685     write_EEPROM(address+1, data);         // 下位バイトの書き込み
686     write_EEPROM(address, data >> 8);     // 上位バイトの書き込み
687 }
688
689
690 /*** EEPROM からの byte 型データの読み出し ***/
691 unsigned char read_EEPROM(unsigned char address)
692 {
693     EEADR = address;           // アドレス設定
694     EECON1bits.EEPGD = 0;      // EEPROM データメモリ選択
695     EECON1bits.CFGS = 0;
696     EECON1bits.RD = 1;         // 読み出し要求
697     while (EECON1bits.RD == 1) // 読み出し終了まで待つ
698     {
699     }
700     return(EEDATA);
701 }
702
703
704 /*** EEPROM への byte 型データの書き込み ***/
705 void write_EEPROM(unsigned char address, unsigned char data)
706 {
707     EEADR = address;           // アドレス設定
708     EEDATA = data;             // 書き込みデータ設定
709     EECON1bits.EEPGD = 0;      // EEPROM データメモリ選択
710     EECON1bits.CFGS = 0;
711     EECON1bits.WREN = 1;       // 書き込み許可
712     EECON2 = 0x55;             // 書き込み手順
713     EECON2 = 0xAA;
714     EECON1bits.WR = 1;         // 書き込み要求
715     while (EECON1bits.WR == 1) // 書き込み終了まで待つ
716     {
717     }
718     EECON1bits.WREN = 0;
719 }

```